



GOLD · KOI 2024 유형 연습

도시 연결망

다익스트라 + 충전소 할인

고등부 1번 — 최단경로 변형 문제 — C언어 완전 설명서

이 설명서의 목표

- 그래프(도시와 도로)를 C 배열로 저장하는 법을 이해한다
- 다익스트라 알고리즘의 원리를 "가장 싼 것부터 확정" 전략으로 이해한다
- 최소 힙(우선순위 큐)을 배열로 직접 만들어본다
- 충전소 조건을 다익스트라에 추가하는 방법을 익힌다
- 예제를 한 단계씩 추적하며 코드 동작을 확인한다

필요한 선행 지식



PART 1

문제 이해
그래프란 뭔가
인접 리스트 저장

PART 2

다익스트라 원리
최소 힙 만들기
충전소 처리

PART 3

전체 코드
예제 추적
직접 풀어보기

GOLD

다익스트라

그래프

최소 힙



PART 1 문제 이해하기

문제를 읽어보자

문제 요약

N개의 도시와 M개의 **단방향** 도로가 있다.

도시 S에서 E까지 이동하는 **최소 비용**을 구한다.

이동 중 **충전소**가 있는 도시에 도착하면, 지금까지 쌓인 비용이 **절반**으로 줄어든다. (소수점 이하 버림)

충전소는 여러 번 이용할 수 있다.

입력 형식

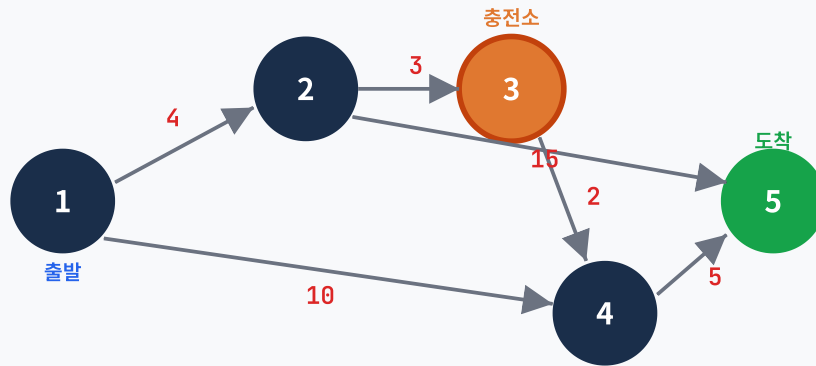
입력stdin

```
// 첫째 줄: 도시 수 N, 도로 수 M, 출발 S, 도착 E
5 6 1 5
// 둘째 줄: 충전소 수 K, 이어서 충전소 도시 번호 K개
1 3
// 다음 M줄: 도로 (출발 도착 비용)
1 2 4
2 3 3
3 4 2
4 5 5
1 4 10
2 5 15
```

제약 조건

항목	범위	의미
N (도시 수)	2 ~ 100,000	도시가 최대 10만 개
M (도로 수)	1 ~ 300,000	도로가 최대 30만 개
K (충전소 수)	0 ~ N	충전소가 없을 수도 있다
c (도로 비용)	1 ~ 10,000	비용은 항상 양수

예제 그래프를 그림으로 보자



주황색 원이 **충전소**(도시 3)이다. 여기에 도착하면 누적 비용이 절반으로 줄어든다.

녹색 원이 **도착지**(도시 5)이다. 출발지는 도시 1이다.

핵심 포인트

"단방향"이라는 건, 1에서 2로 가는 도로가 있다고 해서 2에서 1로 갈 수 있는 게 아니라는 뜻이다.
일방통행 도로라고 생각하면 된다.

PART 1 그래프 저장하기

그래프를 컴퓨터에 어떻게 저장할까?

도시와 도로의 관계를 저장하는 방법은 여러 가지가 있지만, 도시가 최대 10만 개이므로 **인접 리스트**를 사용한다.

인접 리스트란?

각 도시마다 "이 도시에서 갈 수 있는 도시 목록"을 따로 만드는 방식이다.

도시 1의 목록: [(도시2, 비용4), (도시4, 비용10)]

도시 2의 목록: [(도시3, 비용3), (도시5, 비용15)]

도시 3의 목록: [(도시4, 비용2)]

도시 4의 목록: [(도시5, 비용5)]

도시 5의 목록: (없음)

이렇게 하면 메모리도 적게 쓰고, 특정 도시에서 갈 수 있는 곳만 빠르게 확인할 수 있다.

C 코드: 인접 리스트 저장

C에서는 연결 리스트 대신 **배열 3개**로 인접 리스트를 만든다. 이게 가장 빠르고 간단하다.

C

인접 리스트 구조

```
#define MAXN 100001
#define MAXM 300001

// head[u] = 도시 u에서 나가는 간선 중 "첫 번째" 간선 번호
int head[MAXN];

// to[i]    = i번 간선의 도착 도시
// w[i]     = i번 간선의 비용
// nxt[i]   = i번 간선과 같은 출발 도시를 가진 "다음" 간선 번호
int to[MAXM], w[MAXM], nxt[MAXM];
int cnt = 0; // 간선 개수 카운터

// 간선 추가 함수: u에서 v로, 비용 c
void add_edge(int u, int v, int c) {
```

```

cnt++;           // 새 간선 번호
to[cnt] = v;     // 도착지 저장
w[cnt] = c;      // 비용 저장
nxt[cnt] = head[u]; // 기존 첫 간선을 뒤로 밀기
head[u] = cnt;   // 새 간선을 첫 번째로
}

```

head[u] — 도시 **u**에서 나가는 간선들의 시작점이다. 처음엔 0(없음)이다.

to[i], w[i] — **i**번 간선이 어디로 가는지, 비용이 얼마인지 저장한다.

nxt[i] — 같은 도시에서 나가는 다른 간선을 연결한다. 체인처럼 엮는 거다.

add_edge — 새 간선을 맨 앞에 끼워넣는다. 기존 목록 앞에 새 것을 붙이는 방식이다.

비유하면

head[1]은 "도시 1의 전화번호부 첫 페이지"이고,
nxt[i]는 "다음 페이지"라고 생각하면 된다.
 새 연락처(간선)를 추가할 때마다 맨 앞 페이지에 끼워넣는 거다.

도시 **u**에서 나가는 모든 간선 순회

C

간선 순회

```

// 도시 u에서 나가는 모든 도로를 하나씩 확인
for (int e = head[u]; e != 0; e = nxt[e]) {
    int v = to[e]; // 도착 도시
    int c = w[e];  // 비용
    // v까지 비용 c로 갈 수 있다!
}

```

e = head[u] 로 첫 간선부터 시작해서, **e = nxt[e]** 로 다음 간선으로 이동한다. **e** 가 0이면 더 이상 간선이 없다는 뜻이다.

PART 2 다익스트라 알고리즘

가장 싼 길부터 확정하는 전략

다익스트라 알고리즘은 "지금까지 가장 적은 비용으로 갈 수 있는 도시부터 확정한다"는 전략이다.

다익스트라의 핵심 아이디어

1. 출발지의 비용을 0으로 놓고, 나머지는 모두 무한대로 놓는다.
2. 비용이 가장 작은 도시를 하나 꺼낸다.
3. 그 도시에서 갈 수 있는 이웃 도시들의 비용을 갱신한다.
4. 2~3을 반복한다. 도착지를 꺼내면 끝!

"비용이 가장 작은 도시를 꺼내는" 작업이 핵심인데, 이것을 빠르게 하려면 **최소 힙**이 필요하다.

최소 힙이란?

최소 힙은 "가장 작은 값을 빠르게 꺼낼 수 있는 자료구조"이다.

넣기(push)와 꺼내기(pop) 모두 $O(\log N)$ 으로 빠르다.

비유하면

손님이 번호표를 뽑아서 대기하는 은행을 떠올려보자.

일반 줄은 먼저 온 순서대로 부르지만, 최소 힙은 **번호가 가장 작은 사람**을 먼저 부른다.

새 손님이 와도 자동으로 정렬되어 항상 가장 작은 번호가 맨 앞에 있다.

C 코드: 최소 힙

C

최소 힙 구현

```
typedef struct {
    long long cost; // 비용
    int node;      // 도시 번호
} Node;

Node heap[MAXM * 2];
int heap_size = 0;
```

```

// 힙에 넣기: 맨 끝에 넣고 위로 올린다
void push(long long c, int n) {
    int i = ++heap_size;
    heap[i].cost = c;
    heap[i].node = n;
    // 부모보다 작으면 위로 올린다 (swap)
    while (i > 1 && heap[i].cost < heap[i/2].cost) {
        Node tmp = heap[i];
        heap[i] = heap[i/2];
        heap[i/2] = tmp;
        i /= 2;
    }
}

// 힙에서 꺼내기: 맨 위(최솟값)를 꺼내고 재정렬
Node pop() {
    Node top = heap[1];           // 최솟값 저장
    heap[1] = heap[heap_size--]; // 맨 끝을 맨 위로
    int i = 1;
    while (1) {
        int s = i; // 가장 작은 자식 찾기
        if (2*i ≤ heap_size && heap[2*i].cost < heap[s].cost) s = 2*i;
        if (2*i+1 ≤ heap_size && heap[2*i+1].cost < heap[s].cost) s = 2*i+1;
        if (s == i) break; // 더 작은 자식 없으면 끝
        Node tmp = heap[i];
        heap[i] = heap[s];
        heap[s] = tmp;
        i = s;
    }
    return top;
}

```

push — 새 값을 맨 끝에 넣고, 부모보다 작으면 자리를 바꾸며 올라간다.

pop — 맨 위(가장 작은 값)를 꺼낸다. 맨 끝 값을 맨 위로 올리고, 자식보다 크면 내려보낸다.

이 두 연산 모두 트리 높이만큼만 반복하므로 $O(\log N)$ 이다.

PART 2 충전소 처리

이 문제의 핵심: 충전소에서 비용 절반

일반 다익스트라에서는 이웃 도시에 도착할 때 비용을 이렇게 계산한다:

C

일반 다익스트라

```
long long new_cost = dist[u] + c; // 현재까지 비용 + 도로 비용
if (new_cost < dist[v]) {
    dist[v] = new_cost;           // 더 싸면 갱신
}
```

이 문제에서는 **딱 한 줄만** 추가하면 된다:

C

충전소 추가

```
long long new_cost = dist[u] + c;
if (charge[v]) new_cost /= 2; // *** 충전소면 비용 절반! ***
if (new_cost < dist[v]) {
    dist[v] = new_cost;
}
```

왜 이렇게 간단할까?

충전소에 도착하면 비용이 줄어든다. 줄어든 비용은 항상 0 이상이다.

그리고 같은 도시에 비용 10으로 도착하는 것과 비용 12로 도착하는 것 중,

10이 항상 더 유리하다 (이후 어떤 경로를 가든).

그래서 `dist[v]`를 최솟값으로 갱신하기만 하면 다익스트라가 올바르게 동작한다.

charge 배열은 이렇게 만든다

C

충전소 입력

```
int charge[MAXN] = {0}; // 0이면 일반, 1이면 충전소

int K;
```

```
scanf("%d", &K);
for (int i = 0; i < K; i++) {
    int x;
    scanf("%d", &x);
    charge[x] = 1;           // 이 도시는 충전소!
}
```

다익스트라 전체 함수

C

dijkstra()

```
#define INF 1e18    // 아주 큰 수 (도달 불가능)

long long dist[MAXN]; // 각 도시까지의 최소 비용

void dijkstra(int S) {
    // 1단계: 모든 비용을 무한대로 초기화
    for (int i = 1; i ≤ N; i++) dist[i] = INF;
    dist[S] = 0;           // 출발지 비용은 0
    heap_size = 0;
    push(0, S);           // 출발지를 힙에 넣기

    // 2단계: 힙이 빌 때까지 반복
    while (heap_size > 0) {
        Node cur = pop(); // 가장 비용이 작은 도시 꺼내기
        int u = cur.node;

        // 이미 더싼 비용으로 처리됐으면 건너뛰기
        if (cur.cost > dist[u]) continue;

        // 3단계: u에서 갈 수 있는 모든 이웃 확인
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e];
            long long nc = dist[u] + w[e]; // 새 비용

            if (charge[v]) nc /= 2;        // 충전소면 절반!

            if (nc < dist[v]) {
                dist[v] = nc;             // 갱신
                push(nc, v);              // 힙에 넣기
            }
        }
    }
}
```

```
}  
}
```

1단계 — 출발지는 0, 나머지는 무한대(아직 모른다는 뜻)로 시작한다.

2단계 — 힙에서 가장 싼 도시를 꺼내서 처리한다.

continue — 같은 도시가 힙에 여러 번 들어갈 수 있다. 이미 더 싸게 처리됐으면 무시한다.

3단계 — 이웃 도시들의 비용을 계산하고, 더 싸면 갱신한다. 충전소면 $/2$ 가 핵심이다.

PART 3 전체 코드

완성 코드 (복사해서 제출할 수 있다)

```

C city_network.c

#include <stdio.h>

#define MAXN 100001
#define MAXM 300001
#define INF (long long)1e18

typedef struct { long long cost; int node; } Node;

/* ---- 인접 리스트 ---- */
int head[MAXN], to[MAXM], w[MAXM], nxt[MAXM], cnt;

void add(int u, int v, int c) {
    cnt++; to[cnt]=v; w[cnt]=c;
    nxt[cnt]=head[u]; head[u]=cnt;
}

/* ---- 최소 힙 ---- */
Node H[MAXM*2]; int hs;

void push(long long c, int n) {
    int i=++hs; H[i].cost=c; H[i].node=n;
    while(i>1 && H[i].cost<H[i/2].cost)
        { Node t=H[i]; H[i]=H[i/2]; H[i/2]=t; i/=2; }
}

Node pop() {
    Node r=H[1]; H[1]=H[hs--]; int i=1;
    while(1){int s=i;
        if(2*i≤hs && H[2*i].cost<H[i].cost)
            if(2*i+1≤hs && H[2*i+1].cost<H[2*i].cost)
                s=2*i+1;
            if(s==i)break;
        Node t=H[i]; H[i]=H[s]; H[s]=t; i=s;}
    return r;
}

/* ---- 전역 변수 ---- */
int N, M, S, E, K;

```

```

int charge[MAXN];
long long dist[MAXN];

/* ---- 다익스트라 ---- */
void dijkstra() {
    for(int i=1;i≤N;i++) dist[i]=INF;
    dist[S]=0; hs=0; push(0,S);

    while(hs>0){
        Node cur=pop();
        int u=cur.node;
        if(cur.cost>dist[u]) continue;

        for(int e=head[u]; e; e=nxt[e]){
            int v=to[e];
            long long nc=dist[u]+w[e];
            if(charge[v]) nc≠2;      // 충전소 할인
            if(nc<dist[v]) push(nc,v);
        }
    }
}

/* ---- main ---- */
int main() {
    scanf("%d%d%d%d",&N,&M,&S,&E);
    scanf("%d",&K);
    for(int i=0;i<M;i++){
        int x; scanf("%d",&x);
        charge[x]=1;
    }
    for(int i=0;i<M;i++){
        int u,v,c; scanf("%d%d%d",&u,&v,&c);
        add(u,v,c);
    }
    dijkstra();
    if(dist[E]≥INF) printf("-1\n");
    else printf("%lld\n",dist[E]);
    return 0;
}

```

코드 구조 요약

1. **add()** — 도로 정보를 인접 리스트에 저장
2. **push()** / **pop()** — 최소 힙으로 가장 싼 도시를 빠르게 꺼내기

3. **dijkstra()** — 가장 싼 도시부터 확정하며, 충전소면 /2 적용

4. **main()** — 입력 받고 다익스트라 실행 후 결과 출력

PART 3 예제 추적

예제를 한 단계씩 따라가보자

입력: N=5, M=6, S=1, E=5, 충전소=도시3

dist = [_, 0, INF, INF, INF, INF] (1번부터 시작, _는 안 씀)

단계	꺼낸 도시 (비용)	이웃 확인	dist 변화
1	1 (0)	1 → 2: 0+4=4 1 → 4: 0+10=10	dist[2]=4, dist[4]=10
2	2 (4)	2 → 3: 4+3=7, 충전소! 7/2=3 2 → 5: 4+15=19	dist[3]=3, dist[5]=19
3	3 (3)	3 → 4: 3+2=5 (10보다 작다!)	dist[4]=10 → 5
4	4 (5)	4 → 5: 5+5=10 (19보다 작다!)	dist[5]=19 → 10
5	4 (10)	이미 dist[4]=5로 처리됨 → 건너뛰기	변화 없음
6	5 (10)	도착! 5에서 나가는 간선 없음	최종: dist[5] = 10

최적 경로

1 → 2 (비용 4) → 3 (비용 7, 충전소에서 3으로 감소) → 4 (비용 5) → 5 (비용 10)

충전소(도시 3)를 경유해서 비용 7이 3으로 줄었기 때문에,
직행 1 → 4(10) → 5(15)보다 훨씬 싸다.

만약 충전소가 없었다면?

경로	비용 (충전소 없을 때)	비용 (충전소 있을 때)
1 → 2 → 3 → 4 → 5	4+3+2+5 = 14	4+3=7 → 3(충전) +2+5 = 10
1 → 4 → 5	10+5 = 15	10+5 = 15 (충전소 안 거침)
1 → 2 → 5	4+15 = 19	4+15 = 19

충전소를 거치면 비용이 14에서 10으로 줄어든다.

"돌아가더라도 충전소를 거치는 게 이득"인 경우가 이 문제의 핵심이다.

시간 복잡도

얼마나 빠른가?

다익스트라 + 최소 힙 = $O((N + M) \log N)$

$N=100,000$, $M=300,000$ 이면 약 400만 번 연산. 2초 안에 충분하다.

PART 3 직접 풀어보기

연습 문제

아래 입력에 대해 답을 직접 구해보자. 힌트: 충전소를 전략적으로 이용해야 한다.

입력	출력
4 5 1 4 2 2 3 1 2 6 1 3 10 2 3 2 3 4 4 2 4 20	? (직접 계산해보자)

풀이 과정을 여기에 적어보자

각 단계별로 dist 배열이 어떻게 변하는지 적어보자...

예시:

단계 1: 도시 1(비용 0) 꺼냄

- 1->2: 비용 6, 도시2는 충전소이므로 $6/2=3$

- 1->3: 비용 10, 도시3은 충전소이므로 $10/2=5$

핵심 정리

개념	한줄 요약
그래프 저장	인접 리스트 = head + to + w + nxt 배열 4개
다익스트라	가장 싼 도시부터 꺼내서 이웃 비용 갱신, 최소 힙 사용
충전소 처리	도착 도시가 충전소면 <code>new_cost /= 2</code> 한 줄 추가
건너뛰기	<code>if (cur.cost > dist[u]) continue;</code> 로 중복 방지
시간 복잡도	$O((N+M) \log N)$ — 10만 도시도 2초 안에 해결

자주 하는 실수

1. `dist`를 `int`로 선언하면 오버플로우 난다. **반드시** `long long` 사용!
2. 충전소 처리를 `dist` 갱신 **전에** 해야 한다. 갱신 후에 하면 비교가 틀어진다.
3. 도달 불가능한 경우(`dist[E] >= INF`) 반드시 `-1` 출력!
4. `head` 배열 초기값은 0이어야 한다. 전역으로 선언하면 자동으로 0.